Merged by
Soham

Content Management Systems

P Sreenivas MCA

Department of Computer Applications

srinivas@pes.edu

+91 80 2672 1983 Extn 228

PES UNIVERSITY

Content Management Systems

Creating a controller for custom pages

P Sreenivas

Department of Computer Applications

Content Management Systems

Creating a controller for custom pages

A controller is a class that contains a method used to build a page when Drupal is accessed at a specific path. Creating custom pages allows you to extend Drupal beyond just the content pages.

Technical requirements

This requires an existing custom module installed on your Drupal site. This custom module will contain the controllers created throughout this all the steps.

Content Management Systems

Creating a controller for custom pages

PES UNIVERSITY

Defining a controller to provide a custom page

Whenever an HTTP request is made to a Drupal site, the path for the URL is routed to a controller. Controllers are responsible for returning the response for a path that has been defined as a route. Generally, these controllers return render arrays for Drupal's render system to convert into HTML.

Creating a controller for custom pages

How to do it…

1. First, we need to create the src/Controller directory in the module's directory. We will put our controller class in this directory, which gives our controller class the Controller namespace:

```
mkdir -p src/Controller
```

2. Create a file named HelloWorldController.php in the controller directory. This will hold our HelloWorldController controller class.

3. Our HelloWorldController class will extend the ControllerBase base class provided by Drupal core:

```
<?php
namespace Drupal\mymodule\Controller;
use Drupal\Core\Controller\ControllerBase;
class HelloWorldController extends ControllerBase {
}
```

Creating a controller for custom pages

Following PSR-4 autoloading conventions,
our class is in the \Drupal\mymodule\
Controller namespace, which Drupal is able to determine as the src/Controller directory in our module.

As per PSR-4, the filename and class name must also be the same.

The \Drupal\Core\Controller\ControllerBase class provides a handful of utility methods that could be leveraged.

4. Next, we will create a method that returns a render array to display a string of text. Add the following method to our HelloWorldController class:

Content Management Systems

Creating a controller for custom pages

```
/**
 * Returns markup for our custom page.
 *
 * @returns array
 *   The render array.
 */
public function page(): array {
  return [
    '#markup' => '<p>Hello world!</p>'
  ];
}
```

Creating a controller for custom pages

The page method returns a render array that the Drupal rendering system will parse. The #markup key denotes a value that does not have any additional rendering or theming and allows basic HTML elements.

5. Create the mymodule.routing.yml file in your module's directory. The routing.yml file is provided by modules to define routes.

6. The first step in defining a route is to provide a name for the route, which is used as its identifier for URL generation and other purposes:

Creating a controller for custom pages

mymodule.hello_world

7. Give the route a path:
mymodule.hello_world:
path: /hello-world

8. Next, we register the path with our controller. This is done using the defaults key where we provide the controller and page title:

```
mymodule.hello_world:
path: /hello-world
defaults:
_controller: Drupal\mymodule\Controller\
HelloWorldController::page
_title: 'Hello world!‘
```

The _controller key is the fully qualified class name with the class method to be used.
The _title key provides the page title to be displayed

Creating a controller for custom pages

Lastly, define a requirements key to specify the access requirements:

```
mymodule.hello_world:
path: /hello-world
defaults:
_controller: Drupal\mymodule\Controller\
HelloWorldController::page
_title: 'Hello world!'
requirements:
_permission: 'access content‘
```

The _permission option tells the routing system to verify the current user has specific permission in order to view a page.

10. Drupal caches its routing information. We must rebuild Drupal’s caches in order for it to be aware of the module’s new route:

PES UNIVERSITY

Content Management Systems

Using Route Parameters for custom controller

P Sreenivas

Department of Computer Applications

Content Management Systems

Using Route parameters

In the previous example, we defined a route and controller that responded to the /hello-world path. A route can add variables to the path, called route parameters, so that the controller can handle different URLs.

In this step, we will create a route that takes a user ID as a route parameter to display information about that user.

Using Route parameters

Getting started

This step uses the route created in the Defining a controller recipe to provide a custom page.

How to do it…

1. First, remove the src/Routing/RouteSubscriber.php file and mymodule.services.yml from the previous section and clear the Drupal cache, so it does not interfere with what we are about to do.

2. Next, edit routing.yml so that we can add a route parameter of user to the path:

Content Management Systems

Using Route parameters

path: /hello-world/{user}

Route parameters are wrapped with an opening bracket ({) and a closing bracket (})

Next, we will update the requirements key to specify that the user parameter should be an integer for a user ID, and that the current user has access to view other profiles:

```
requirements:
user: '\d+'
_entity_access: 'user.view‘
```

The user key under requirements represents the same route parameter. The routing system allows you to provide regular expressions to validate the value of a route parameter. This validates the user parameter as any digit value.

Using Route parameters

We then use the _entity_access requirement to verify the current user has access to perform the view operation on the entity type user.

4. Then, we add a new options key to the route definition so that we can identify what type of value the user route parameter is:

```
options:
parameters:
user:
type: 'entity:user'
```

Using Route parameters

This defines the user route parameter as being of the entity:user type. Drupal will use this to convert the ID value into a user object for our controller.

5. Open the src/Controller/HelloWorldController.php file so that the page method can be updated to receive the user route parameter value.

6. Update the page method to receive the user route parameter as a user object in its method parameters:

Content Management Systems

Using Route parameters

```
/**
 * Returns markup for our custom page.
 *
 * @param \Drupal\user\UserInterface $user
 *   The user parameter.
 *
 * @returns array
 *   The render array.
 */
public function page(UserInterface $user): array {
  return [
    '#markup' => '<p>Hello world!</p>'
  ];
}
```

Using Route parameters

The routing system passes route parameter values to the method based on parameter names for the method. Our parameter's definition specifies that Drupal should convert the ID in the URL into a user object.

7. Let's adjust the returned text from Hello world! to replace "world" with the email of the user object:

```
return [
'#markup' => sprintf('<p>Hello %s!</p>',
$user->getEmail()),
];
```

Using Route parameters

We use sprintf to return a formatted string that contains the user's email address.

8. Drupal caches its routing information. We must rebuild Drupal's caches in order for it to be aware of the changes to the module's new route:

php vendor/bin/drush cr

9. When you visit /hello-world, the route's old path without a route parameter, Drupal will return a 404 error for page not found.

10. When you visit /hello-world/1, the page will display our formatted text with the user's email address:

Content Management Systems

Using Route parameters

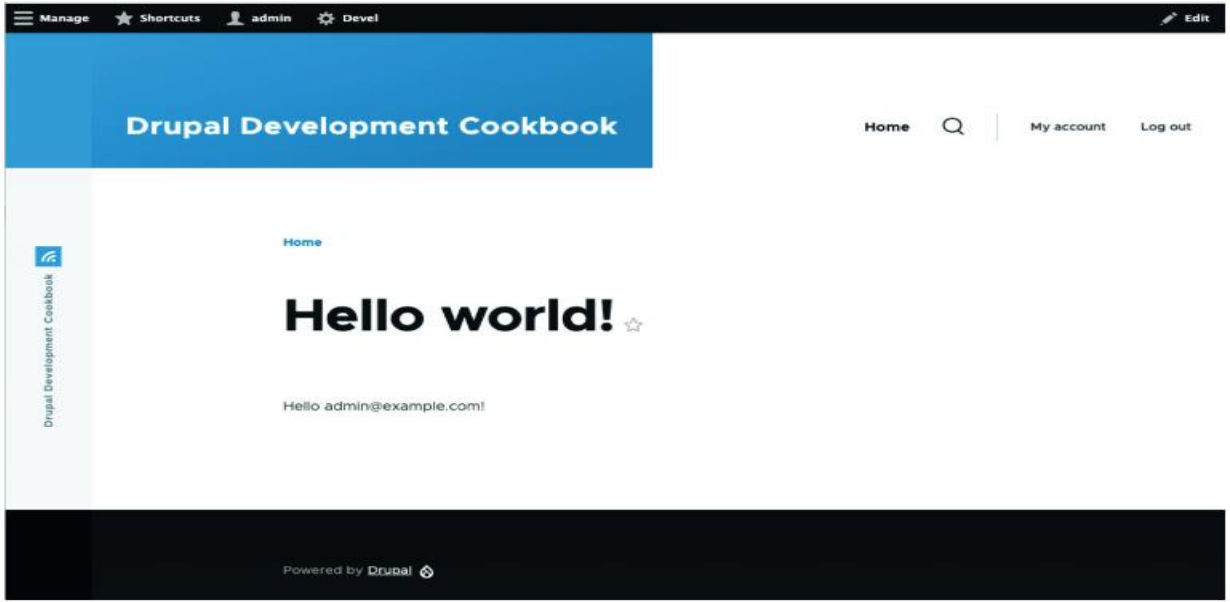

Figure 5.2 – The formatted text with the user's email address

PES UNIVERSITY

Content Management Systems

Creating a dynamic redirect page

P Sreenivas

Department of Computer Applications

Content Management Systems

Creating a dynamic redirect page

Controllers for routes in Drupal can return request objects provided by the Symfony HttpFoundation component instead of a render array. When a response object is returned, the render system is bypassed and the response object is handled directly.

In this step, we will create a route that redirects authenticated users to the homepage and anonymous users to the user login form.

Creating a dynamic redirect page

1. First, we need to create the src/Controller directory in the module's directory. We will put our controller class in this directory, which gives our controller class the Controller namespace:

mkdir -p src/Controller

2. Create a file named RedirectController.php in the Controller directory. This will hold our RedirectController controller class.

3. Our RedirectController class will extend the ControllerBase base class provided by Drupal core:

Content Management Systems

Creating a dynamic redirect page

```
<?php

namespace Drupal\mymodule\Controller;

use Drupal\Core\Controller\ControllerBase;

class RedirectController extends ControllerBase {

}
```

Creating a dynamic redirect page

Following PSR-4 autoloading conventions, our class is in the \Drupal\mymodule\Controller namespace, which Drupal can determine as the src/Controller directory in our module. As per PSR-4, the filename and class name must also be the same.

The \Drupal\Core\Controller\ControllerBase class provides a handful of utility methods that can be leveraged.

4. Next, we will create a method that checks whether the current user is logged in or not. Update the RedirectController class to include the following method:

Content Management Systems

Creating a dynamic redirect page

```
<?php

namespace Drupal\mymodule\Controller;

use Drupal\Core\Controller\ControllerBase;
use Symfony\Component\HttpFoundation\RedirectResponse;
```

Content Management Systems

Creating a dynamic redirect page

```
class RedirectController extends ControllerBase {

  /**
   * Returns redirect to home or user login form.
   *
   * @return \Symfony\Component\HttpFoundation\
       RedirectResponse
   *   The redirect response.
   */
  public function page(): RedirectResponse {
    if ($this->currentUser()->isAuthenticated()) {
      // Redirect to the homepage.
    }
    else {
      // Redirect to the user login form.
    }
  }
}
```

Creating a dynamic redirect page

The ControllerBase class provides a currentUser method that accesses the current user object. We can call the isAuthenticated method to check whether the user is logged in or not.

5. Next, we will update the code to return a RedirectResponse object to a URL that is generated from a route name:

Content Management Systems

Creating a dynamic redirect page

```
public function page(): RedirectResponse {
  if ($this->currentUser()->isAuthenticated()) {
    $route_name = '<front>';
  }
  else {
    $route_name = 'user.login';
  }
  $url = \Drupal\Core\Url::fromRoute($route_name);
  return new RedirectResponse($url->toString());
}
```

Creating a dynamic redirect page

We set the route name in the $route_name variable. While <front> is not a typical path, it is a route name available to route to the appropriate path that is set as the homepage. Drupal core defines special slugs such as <front> and <none> for convenience in routing.

We then construct a new URL object with the static fromRoute method of the \Drupal\ Core\Url class. We then return a RedirectResponse object that redirects to our URL. We have to get the string for the URL using the toString method on the URL object.

Creating a dynamic redirect page

6. Create the mymodule.routing.yml file in your module's directory. The routing.yml file is provided by modules to define routes.

7. The first step in defining a route is to provide a name for the route, which is used as its identifier for URL generation and other purposes:

mymodule.user_redirect

Creating a dynamic redirect page

8. Give the route a path:
mymodule.user_redirect:
path: /user-redirect

9. Next, we register the path with our controller. This is done with the defaults key where we provide the controller:

```
mymodule.user_redirect:
path: /user-redirect
defaults:
_controller: Drupal\mymodule\Controller\
RedirectController::page
```

The _controller key is the fully qualified class name with the class method to be used.

Creating a dynamic redirect page

10. Lastly, define a requirements key to specify the access requirements. We want the route to always be accessible since we handle authenticated and anonymous users:

```
mymodule.user_redirect:
path: /user-redirect
defaults:
_controller: Drupal\mymodule\Controller\
RedirectController::page
requirements:
_access: 'TRUE‘
```

The _access option allows us to specify that a route is always accessible.

Creating a dynamic redirect page

11. Drupal caches its routing information. We must rebuild Drupal's caches in order for it to be aware of the module's new route:
php vendor/bin/drush cr

12. Go to /user-redirect on your Drupal site as an anonymous user and you will be redirected to the user login form. If you are authenticated, you will be redirected to the homepage.

Creating a dynamic redirect page

How it works…

The RedirectResponse object comes from Symfony's HttpFoundation component and represents an HTTP redirect response. When converted into a response sent from the web server, it will set the location header to the URL provided. It also passes one of the valid 3xx HTTP status codes.

The redirect response defaults to the HTTP status code of 302 Found. A 302 redirect is intended to be temporary and not cacheable by the browser.

Drupal can generate URLs from a given route name.

PES UNIVERSITY

Content Management Systems

Creating a JSON Response

P Sreenivas

Department of Computer Applications

Creating a JSON response

Routes can also return JavaScript Object Notation (JSON) responses. A JSON response is often used for building API routes since it is an interchange format that is supported by all programming languages.

This allows exposing data to be consumed by a third-party consumer.

In this process, we will create a route that returns a JSON response containing information about the Drupal site or user details.

Creating a JSON response

Step 1: Create a Custom Module

Let’s call it json_response_demo.

File structure:

```
json_response_demo/
├── json_response_demo.info.yml
├── json_response_demo.routing.yml
└── src/Controller/JsonResponseDemoController.php
```

Creating a JSON response

Step 2: Define the Module Info File

File: json_response_demo.info.yml

```
name: 'JSON Response Demo'
type: module
description: 'Demonstrates returning JSON response from a controller.'
core_version_requirement: ^10
package: Custom
```

Creating a JSON response

Step 3: Define a Route

File: json_response_demo.routing.yml

```
json_response_demo.example:
  path: '/json-demo'
  defaults:
    _controller:
'\Drupal\json_response_demo\Controller\JsonResponseDemoController::content'
    _title: 'JSON Response Example'
  requirements:
    _permission: 'access content'
```

Creating a JSON response

Step 4: Create the Controller

File: src/Controller/JsonResponseDemoController.php

```
<?php
 namespace Drupal\json_response_demo\Controller;
use Symfony\Component\HttpFoundation\JsonResponse;
use Drupal\Core\Controller\ControllerBase;

class JsonResponseDemoController extends ControllerBase {

  /**
   * Returns a JSON response.
   */
```

Creating a JSON response

```
public function content() {
    // Create some sample data
    $data = [
      'status' => 'success',
      'message' => 'This is a JSON response from Drupal 10!',
      'data' => [
        'user' => 'John Doe',
        'role' => 'editor',
        'timestamp' => \Drupal::time()->getCurrentTime(),
      ],
    ];

    // Return as JSON
    return new JsonResponse($data);
  }
}
```

Creating a JSON response

Step 5: Enable and Test

Enable your module:

drush en json_response_demo

or

Extend→custom→json_response_demo, select the module and install the same

Then open the URL in your browser:

http://your-site.local/json-demo

You should see a JSON output like:


```
{
  "status": "success",
  "message": "This is a JSON response from Drupal 10!",
  "data": {
    "user": "John Doe",
    "role": "editor",
    "timestamp": 1730630400
  } }
```

PES UNIVERSITY

Content Management Systems

Creating a Custom Form and saving configuration changes

P Sreenivas

Department of Computer Applications

Content Management Systems

Creating a Custom form

The step-by-step instructions for creating custom form in Drupal 10 is as follows:

.Folder Structure

```
C:\xampp\htdocs\drupal\modules\custom\contact_form\
│
├── contact_form.info.yml
├── contact_form.routing.yml
├── contact_form.install
└── src\Form\ContactForm.php
```

Creating a Custom form

Step 2: Create the required files

File 1: contact_form.info.yml

```
name: 'Contact Form'
type: module
description: 'A simple custom contact form that saves data in the database.'
core_version_requirement: ^10
package: Custom
version: 1.0
```

Creating a Custom form

File 2: contact_form.routing.yml

```
contact_form.simple:
  path: '/contact-us'
  defaults:
    _form: '\Drupal\contact_form\Form\ContactForm'
    _title: 'Contact Us'
  requirements:
    _permission: 'access content'
```

Creating a Custom form

File 3: contact_form.install

```
<?php

function contact_form_schema() {
  $schema['contact_form_data'] = [
    'description' => 'Stores contact form submissions.',
    'fields' => [
      'id' => ['type' => 'serial','unsigned' => TRUE,'not null' => TRUE,],
      'name' => ['type' => 'varchar','length' => 100,'not null' => TRUE,],
      'email' => ['type' => 'varchar','length' => 150,'not null' => TRUE,],
      'message' => ['type' => 'text','not null' => TRUE,],
    ],
    'primary key' => ['id'],
  ];
  return $schema;
}
```

Creating a Custom form

File 4: src\Form\ContactForm.php

```
<?php

namespace Drupal\contact_form\Form;

use Drupal\Core\Form\FormBase;
use Drupal\Core\Form\FormStateInterface;
use Drupal\Core\Database\Database;

class ContactForm extends FormBase {
  public function getFormId() { return 'contact_form'; }
```

Content Management Systems

Creating a Custom form

```
public function buildForm(array $form, FormStateInterface $form_state) {
    $form['name'] = ['#type' => 'textfield','#title' => $this->t('Your Name'),'#required' => TRUE,];
    $form['email'] = ['#type' => 'email','#title' => $this->t('Your Email'),'#required' => TRUE,];
    $form['message'] = ['#type' => 'textarea','#title' => $this->t('Your Message'),'#required' => TRUE,];
    $form['submit'] = ['#type' => 'submit','#value' => $this->t('Send Message'),];
    return $form;
  }
```

Content Management Systems

Creating a Custom form

```
 public function submitForm(array &$form, FormStateInterface $form_state) {
    $conn = Database::getConnection();
    $conn->insert('contact_form_data')->fields([
      'name' => $form_state->getValue('name'),
      'email' => $form_state->getValue('email'),
      'message' => $form_state->getValue('message'),
    ])->execute();
    $this->messenger()->addMessage($this->t('Thank you for contacting us!'));
  }
}
```

Creating a Custom form

Step 3: Enable the module

Open your Drupal admin dashboard.
Navigate to Extend → Search for "Contact Form" under Custom → Check and click Install.

Step 4: Test the form

Open the following URL:
http://localhost/drupal/contact-us

Submit the form and verify the message:
Thank you for contacting us!

Creating a Custom form

Step 5: View saved data

Open phpMyAdmin → Select database (drupal) → Click on the table contact_form_data → View all submitted records.

PES UNIVERSITY

Content Management Systems

Specifying conditional form elements in Drupal 10

P Sreenivas

Department of Computer Applications

Specifying conditional form elements

Concept Overview

Conditional form elements allow you to **show, hide, or modify** certain fields **based on user input** in another field.

In **Drupal 10**, this can be done using the **Form API #states property**, which provides **client-side dynamic behavior** (using jQuery under the hood).

Specifying conditional form elements

Key Property: #states

The #states property defines **conditions** under which a form element should change its **visibility**, **required status**, or **enabled/disabled** state.

Basic Syntax:

```
'#states' => [
  'visible' => [
    ':input[name="field_name"]' => ['value' => 'SomeValue'],
  ],
],
```

This means:
The current element is visible when the form field named field_name has the value 'SomeValue'.

Specifying conditional form elements

Example: Conditional Field in a Custom Contact Form

Let’s create a custom form where:

The user selects a **contact reason** (General, Support, Feedback).
If they choose **Support**, an **“Issue Description”** text area appears.

Specifying conditional form elements

Step 1: Create a Custom Form Module
Create a custom module (e.g., custom_contact_form).

File: custom_contact_form.info.yml
name: 'Custom Contact Form'
type: module
description: 'A custom contact form with conditional elements.'
core_version_requirement: ^10
package: Custom

.

Specifying conditional form elements

Step 2: Define the Form Class

File: src/Form/CustomContactForm.php

```
<?php

namespace Drupal\custom_contact_form\Form;

use Drupal\Core\Form\FormBase;
use Drupal\Core\Form\FormStateInterface;

class CustomContactForm extends FormBase {

  public function getFormId() {
    return 'custom_contact_form';
  }
```

Specifying conditional form elements


```
public function buildForm(array $form, FormStateInterface $form_state) {

  // Contact reason (select list)
  $form['contact_reason'] = [
    '#type' => 'select',
    '#title' => $this->t('Reason for Contact'),
    '#options' => [
      'general' => $this->t('General Inquiry'),
      'support' => $this->t('Technical Support'),
      'feedback' => $this->t('Feedback'),
    ],
    '#required' => TRUE,
  ];
```

Content Management Systems

Specifying conditional form elements

```
// Conditional text area
    $form['issue_description'] = [
      '#type' => 'textarea',
      '#title' => $this->t('Describe your issue'),
      '#description' => $this->t('Please describe your technical problem.'),
      '#states' => [
        'visible' => [
          ':input[name="contact_reason"]' => ['value' => 'support'],
        ],
        'required' => [
          ':input[name="contact_reason"]' => ['value' => 'support'],
        ],
      ],
    ];
```

Content Management Systems

Specifying conditional form elements

```
// Common fields
    $form['message'] = [
      '#type' => 'textarea',
      '#title' => $this->t('Message'),
      '#required' => TRUE,
    ];

    $form['actions']['submit'] = [
      '#type' => 'submit',
      '#value' => $this->t('Send Message'),
    ];

    return $form;
  }
```

Specifying conditional form elements

```
public function submitForm(array &$form, FormStateInterface $form_state) {
    \Drupal::messenger()->addStatus($this->t('Your message has been submitted.'));
  }

}
```

Specifying conditional form elements

Step 3: Add Routing

File: custom_contact_form.routing.yml

```
custom_contact_form.form:
  path: '/custom-contact'
  defaults:
    _form: '\Drupal\custom_contact_form\Form\CustomContactForm'
    _title: 'Contact Us'
  requirements:
    _permission: 'access content'
```

Specifying conditional form elements

Step 4: Enable and Test
Run:
drush en custom_contact_form
drush cr

or

Extend→select or check the module → Install the module
Then visit:

- **/custom-contact**

Now, when you select **“Technical Support”**, the **Issue Description** field appears automatically!

PES UNIVERSITY

Content Management Systems

Specifying conditional form elements in Drupal 10

P Sreenivas

Department of Computer Applications

Specifying conditional form elements

Concept Overview

Conditional form elements allow you to **show, hide, or modify** certain fields **based on user input** in another field.

In **Drupal 10**, this can be done using the **Form API #states property**, which provides **client-side dynamic behavior** (using jQuery under the hood).

Specifying conditional form elements

Key Property: #states

The #states property defines **conditions** under which a form element should change its **visibility**, **required status**, or **enabled/disabled** state.

Basic Syntax:

```
'#states' => [
  'visible' => [
    ':input[name="field_name"]' => ['value' => 'SomeValue'],
  ],
],
```

This means:
The current element is visible when the form field named field_name has the value 'SomeValue'.

Specifying conditional form elements

Example: Conditional Field in a Custom Contact Form

Let’s create a custom form where:

The user selects a **contact reason** (General, Support, Feedback).
If they choose **Support**, an **“Issue Description”** text area appears.

Specifying conditional form elements

Step 1: Create a Custom Form Module
Create a custom module (e.g., custom_contact_form).

File: custom_contact_form.info.yml
name: 'Custom Contact Form'
type: module
description: 'A custom contact form with conditional elements.'
core_version_requirement: ^10
package: Custom

.

Specifying conditional form elements

Step 2: Define the Form Class

File: src/Form/CustomContactForm.php

```
<?php

namespace Drupal\custom_contact_form\Form;

use Drupal\Core\Form\FormBase;
use Drupal\Core\Form\FormStateInterface;

class CustomContactForm extends FormBase {

  public function getFormId() {
    return 'custom_contact_form';
  }
```

Specifying conditional form elements


```
public function buildForm(array $form, FormStateInterface $form_state) {

  // Contact reason (select list)
  $form['contact_reason'] = [
    '#type' => 'select',
    '#title' => $this->t('Reason for Contact'),
    '#options' => [
      'general' => $this->t('General Inquiry'),
      'support' => $this->t('Technical Support'),
      'feedback' => $this->t('Feedback'),
    ],
    '#required' => TRUE,
  ];
```

Content Management Systems

Specifying conditional form elements

```
// Conditional text area
    $form['issue_description'] = [
      '#type' => 'textarea',
      '#title' => $this->t('Describe your issue'),
      '#description' => $this->t('Please describe your technical problem.'),
      '#states' => [
        'visible' => [
          ':input[name="contact_reason"]' => ['value' => 'support'],
        ],
        'required' => [
          ':input[name="contact_reason"]' => ['value' => 'support'],
        ],
      ],
    ];
```

Content Management Systems

Specifying conditional form elements

```
// Common fields
    $form['message'] = [
      '#type' => 'textarea',
      '#title' => $this->t('Message'),
      '#required' => TRUE,
    ];

    $form['actions']['submit'] = [
      '#type' => 'submit',
      '#value' => $this->t('Send Message'),
    ];

    return $form;
  }
```

Specifying conditional form elements

```
public function submitForm(array &$form, FormStateInterface $form_state) {
    \Drupal::messenger()->addStatus($this->t('Your message has been submitted.'));
  }

}
```

Specifying conditional form elements

Step 3: Add Routing

File: custom_contact_form.routing.yml

```
custom_contact_form.form:
  path: '/custom-contact'
  defaults:
    _form: '\Drupal\custom_contact_form\Form\CustomContactForm'
    _title: 'Contact Us'
  requirements:
    _permission: 'access content'
```

Specifying conditional form elements

Step 4: Enable and Test
Run:
drush en custom_contact_form
drush cr

or

Extend→select or check the module → Install the module
Then visit:

- **/custom-contact**

Now, when you select **“Technical Support”**, the **Issue Description** field appears automatically!

PES UNIVERSITY

Content Management Systems

Customizing existing forms in Drupal

P Sreenivas

Department of Computer Applications

Content Management Systems

Customizing existing forms in Drupal

The Form API does not just provide a way to create forms. There are ways to alter existing forms through hooks in a custom module. By using this technique, new elements can be added, default values can be changed, and elements can even be hidden from view to simplify the user experience.

The altering of a form does not happen in a custom class; this is a hook defined in the module file. In this recipe, we will use the hook_form_FORM_ID_alter() hook to add a telephone field to the site's configuration form.

Customizing existing forms in Drupal

How to do it…
1. Ensure that your module has a .module file to contain hooks, such as mymodule.module.
2. We will implement hook_form_FORM_ID_alter for the system_site_information_ settings form:

```
<?php
use Drupal\Core\Form\FormStateInterface;
function mymodule_form_system_site_information
_settings_alter(array &$form, FormStateInterface
$form_state) {
// Code to alter form or form state here
}
```

Customizing existing forms in Drupal

Drupal will call this hook and pass the current form array and its form state object. The form array is passed by reference, allowing our hook to modify the array without returning any values. This is why the $form parameter has the ampersand (&) before it.

In PHP, all objects are passed by reference, which is why we have no ampersand before $form_state.

Form IDs can be found by inspecting the getFormId method of the form class.

3. Next, we add our telephone field to the form so that it can be displayed and saved:

Content Management Systems

Customizing existing forms in Drupal

```
<?php

use Drupal\Core\Form\FormStateInterface;

function mymodule_form_system_site_information_
  settings_alter(array &$form, FormStateInterface
    $form_state) {
  $form['site_information']['site_phone'] = [
    '#type' => 'tel',
    '#title' => 'Site phone',
    '#default_value' => \Drupal::config('system.site')
        ->get('phone'),
  ];
}
```

Customizing existing forms in Drupal

We retrieve the current phone value from the system.site configuration object so that it can be modified if already set.

4. We then need to add a submit handler to the form in order to save the configuration for our new field:

Content Management Systems

Customizing existing forms in Drupal

```
<?php
use Drupal\Core\Form\FormStateInterface;
function mymodule_form_system_site_information
_settings_alter(array &$form, FormStateInterface
$form_state) {
$form['site_information']['site_phone'] = [
'#type' => 'tel',
'#title' => 'Site phone',
'#default_value' => \Drupal::config('system.site')
->get('phone'),
];
$form['#submit'][] = 'mymodule_system_site_
information_phone_submit';
}
```

Customizing existing forms in Drupal

```
function mymodule_system_site_information_phone_submit
(array &$form, FormStateInterface $form_state) {
$config = Drupal::configFactory()->
getEditable('system.site');
$config ->set('phone', $form_state->
getValue('site_phone'));
}
```

Customizing existing forms in Drupal

The $form['#submit'] modification adds our callback to the form's submit handlers. This allows our module to interact with the form once it has been submitted.

The mymodule_system_site_information_phone_submit callback is passed the form array and form state. We load the current configuration factory to receive the configuration that can be edited. We then load the system.site configuration object and save phone based on the value from the form state.

5. Rebuild your Drupal site's cache to make it aware of the new hook, so that it will be invoked when viewing the site settings form:

Content Management Systems

Customizing existing forms in Drupal

6. Visit the site settings configuration form at /admin/config/system/site-information:

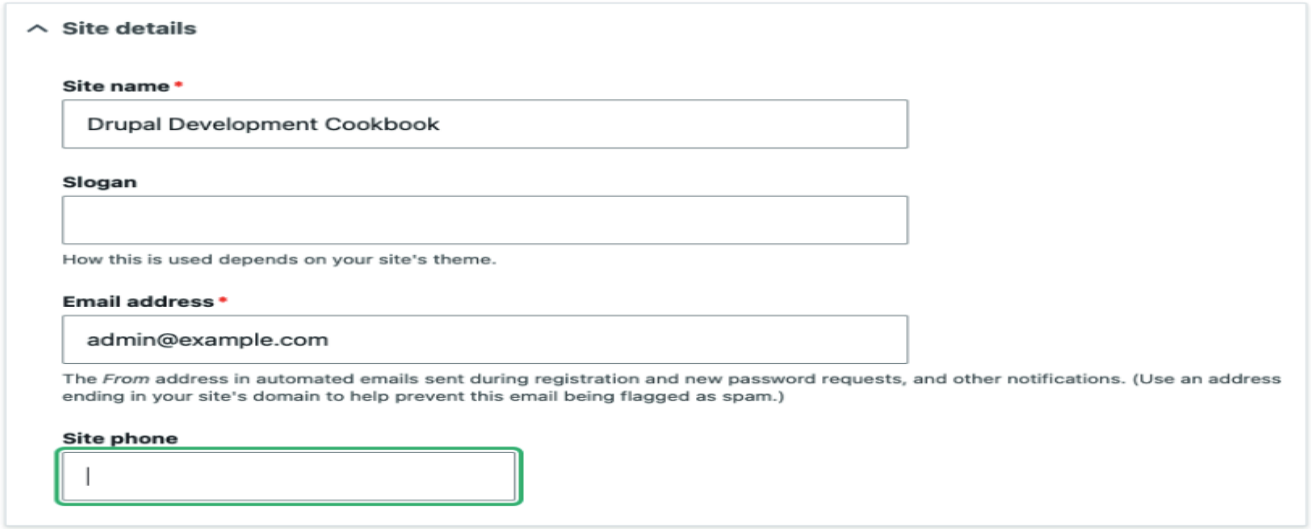

Figure 7.5 – The altered site settings form

Content Management Systems

Creating blocks using plugins

P Sreenivas

Department of Computer Applications

Content Management Systems

Creating blocks using plugins

In Drupal, a block is a piece of content that can be placed in a region provided by a theme. Blocks are used to present specific kinds of content, such as a user login form, a snippet of text, and many more.

Blocks are annotated plugins. Annotated plugins use documentation blocks to provide details of the plugin. They are discovered in the module's Plugin class namespace. Each class in the Plugin/Block namespace will be discovered by the Block module's plugin manager.

In this process, we will define a block that will display the block will display the message "Welcome to PES University Block."

Content Management Systems

Creating blocks using plugins

Folder and File Directory Structure

C:\xampp\htdocs\drupal\modules\custom\custom_block_example

```
custom_block_example.info.yml
src
    Plugin
        Block
            HelloBlock.php
```

Creating blocks using plugins

Step 1: Create the Info File

File: custom_block_example.info.yml

```
name: 'Custom Block Example'
 type: module
 description: 'A simple custom block created using a plugin.'
 core_version_requirement: ^10
 package: Custom
```

Creating blocks using plugins

Step 2: Create the Block PHP File
File: src/Plugin/Block/HelloBlock.php

```
<?php
namespace Drupal\custom_block_example\Plugin\Block;
use Drupal\Core\Block\BlockBase;

/**
 * Provides a 'Hello' Custom Block.
 *
 * @Block(
 *   id = "hello_block",
 *   admin_label = @Translation("Hello Custom Block")
 * )
 */
```

Content Management Systems

Creating blocks using plugins

```
class HelloBlock extends BlockBase {

  /**
   * {@inheritdoc}
   */
  public function build() {
    return [
      '#markup' => $this->t('Welcome to PES University Block!'),
    ];
  }

}
```

Creating blocks using plugins

Step 3: Clear Cache and Enable the Module

Open Drupal in your browser:
http://localhost:8080/admin/config/development/performance
Click **Clear all caches**

Go to:
http://localhost:8080/admin/modules

Or Extend →
Find **Custom Block Example** under the *Custom* section
Check it and click **Install**

Creating blocks using plugins

Step 5: Place the Block

Go to **Structure** → **Block Layout** → **Place Block**

Search for **Hello Custom Block**

Click **Place block** and choose any region (for example, Sidebar or Footer)

Click **Save Blocks**

Creating blocks using plugins

Step 8: Verify

Open your Drupal site in the browser at:

http:localhost/drupal

You will see your custom block displayed as:

Welcome to PES University Block!

Content Management Systems

Creating Custom field type

P Sreenivas

Department of Computer Applications

Content Management Systems

Creating custom field type

Field types are defined using the plugin system. Each field type has its own class and definition. A new field type can be defined through a custom class that will provide schema and property information.

Field types define ways in which data can be stored and handled through the Field API on entities. Field widgets provide means for editing a field type in the user interface. Field formatters provide means for displaying the field data to users. Both are plugins and will be covered in later steps.

In this example, we will create a simple field type called realname to store the first and last names and add it to a comment type.

Content Management Systems

Creating custom field type

1. Understanding Field Types

In Drupal, field types are plugins that define how data is stored, validated, and handled within entities (like nodes, comments, users, etc.).

Each field type has:
- Schema: Defines how data is stored in the database.
- Property definitions: Defines how the field's data is represented in Drupal.
- Widget: Controls how users enter data (input form).
- Formatter: Controls how data is displayed.

Creating custom field type

2. Set Up Your Custom Module
Create a custom module (e.g., realname_field).

```
realname_field/
├── realname_field.info.yml
├── realname_field.module
└── src/
    └── Plugin/
        └── Field/
            └── FieldType/
```

Creating custom field type

Example: realname_field.info.yml

```
name: Real Name Field
type: module
description: Provides a custom field type for first and last names.
core_version_requirement: ^10
package: Custom
```

Creating custom field type

3. Create the Field Type Plugin

Create a file named:
src/Plugin/Field/FieldType/RealName.php

Example: RealName.php

```
<?php

namespace Drupal\realname_field\Plugin\Field\FieldType;

use Drupal\Core\Field\FieldItemBase;
use Drupal\Core\Field\FieldStorageDefinitionInterface;
use Drupal\Core\TypedData\DataDefinition;

/**
```

Creating custom field type


```
* Plugin implementation of the 'realname' field type.
 *
 * @FieldType(
 *   id = "realname",
 *   label = @Translation("Real Name"),
 *   description = @Translation("Stores a user's first and last names."),
 *   category = @Translation("Custom"),
 *   default_widget = "string_textfield",
 *   default_formatter = "string"
 * )
 */
```

Content Management Systems

Creating custom field type

```
class RealName extends FieldItemBase {

  /**
   * {@inheritdoc}
   */
  public static function schema(FieldStorageDefinitionInterface $field_definition) {
    return [
      'columns' => [
        'first_name' => [
          'description' => 'First name.',
          'type' => 'varchar',
          'length' => 255,
          'not null' => TRUE,
          'default' => '',
        ],
```

Content Management Systems

Creating custom field type

```
'last_name' => [
          'description' => 'Last name.',
          'type' => 'varchar',
          'length' => 255,
          'not null' => TRUE,
          'default' => '',
        ],
      ],
      'indexes' => [
        'first_name' => ['first_name'],
        'last_name' => ['last_name'],
      ],
    ];
  }
```

Content Management Systems

Creating custom field type

```
/**
  * {@inheritdoc}
  */
 public static function propertyDefinitions(FieldStorageDefinitionInterface $field_definition) {
   $properties['first_name'] = DataDefinition::create('string')
     ->setLabel(t('First name'));

   $properties['last_name'] = DataDefinition::create('string')
     ->setLabel(t('Last name'));

   return $properties;
 }
 /**
  * {@inheritdoc}
  */
 public static function mainPropertyName() {
   return 'first_name';
 }}
```

Creating custom field type

4. Clear Cache

Once the field type is defined, **rebuild Drupal's cache** so that the system recognizes the new field plugin:

drush cr

Or

Navigate to **Configuration** → **Performance** → **Clear all caches**.

Creating custom field type

5. Add the Field Type to an Entity

Now you can use the field type in the Drupal UI:

Go to **Structure** → **Content types** → **[Your type]** → **Manage fields**.

Click **Add field** → Choose “Real Name”.

Configure and save it.

Creating custom field type

6. Result

You now have a **custom field type** that stores **first name** and **last name** as separate database columns.

Later, you can create:

A **custom widget** (for a combined input field).
A **custom formatter** (to display “First Last”).

PES UNIVERSITY

Content Management Systems

Creating Custom field formatter

P Sreenivas

Department of Computer Applications

Content Management Systems

Creating custom field formatter

Field formatters define the way in which a field type will be presented. These formatters return the render array information to be processed by the theming layer. Field formatters are configured on the display mode interfaces.

In this process, we will create a formatter for the field created in the Creating a custom field type recipe in this chapter. The field formatter will display the first and last name values inline, as a full name.

Content Management Systems

Creating custom field formatter

PES UNIVERSITY

You already have:

a **Field Type** (RealName) → defines how data is stored
a **Field Widget** (RealNameDefaultWidget) → defines how data is entered

Now you'll create:
a **Field Formatter** → defines how the data is *displayed* on the site (e.g., "John Doe").

Content Management Systems

Creating custom field formatter

Folder Structure

Your module (realname_field) should now look like this:

```
realname_field/
├── realname_field.info.yml
└── src/
    └── Plugin/
        └── Field/
            ├── FieldType/
            │   └── RealName.php
            ├── FieldWidget/
            │   └── RealNameDefaultWidget.php
            └── FieldFormatter/
                └── RealNameDefaultFormatter.php
```

Creating custom field formatter

Create the Custom Field Formatter Plugin
Create this file:

src/Plugin/Field/FieldFormatter/RealNameDefaultFormatter.php

```
<?php

namespace Drupal\realname_field\Plugin\Field\FieldFormatter;

use Drupal\Core\Field\FormatterBase;
use Drupal\Core\Field\FieldItemListInterface;
```

Creating custom field formatter


```
/**
 * Plugin implementation of the 'realname_default' formatter.
 *
 * @FieldFormatter(
 *   id = "realname_default",
 *   label = @Translation("Real Name (First + Last)"),
 *   field_types = {
 *     "realname"
 *   }
 * )
 */
class RealNameDefaultFormatter extends FormatterBase {
```

Content Management Systems

Creating custom field formatter

```
 /**
   * {@inheritdoc}
   */
  public function viewElements(FieldItemListInterface $items, $langcode) {
    $elements = [];
foreach ($items as $delta => $item) {
      $first_name = $item->first_name;
      $last_name = $item->last_name;
      $full_name = trim("$first_name $last_name");

      // Render as a simple markup string.
      $elements[$delta] = [
        '#markup' => $full_name,
      ];
    }
     return $elements;
  }  }
```

Creating custom field formatter

Rebuild the Cache

After creating this file, run:
drush cr or Configuration→Performance→Clear all caches
This clears Drupal's plugin cache so your new formatter is discovered.

Use the Formatter

Go to **Structure** → **Content types** → **Article** → **Manage display**
Find your **Real Name** field
In the **Format** dropdown, select **"Real Name (First + Last)"**
Save the display settings
Now, when you view a node, you'll see the formatted name (e.g., John Doe) displayed.

PES UNIVERSITY

Content Management Systems

Creating Custom field widget

P Sreenivas

Department of Computer Applications

Creating custom field widget

Field widgets provide the form component to a field in an entity form. These integrate with the Form API to define how a field can be edited and the way in which the data can be formatted before it is saved.

Field widgets are chosen and customized through the form display interface.

In this process, we will create a widget for the field created in the Creating a custom field type step in this chapter. The field widget will provide two text fields for entering the first and last name items.

Creating custom field widget

PES UNIVERSITY

What Is a Field Widget?

A **field widget** in Drupal defines how the field's data is **entered in the form** — i.e., how the editing interface looks and behaves.

Field type → defines *how data is stored*
Field widget → defines *how users enter data*
Field formatter → defines *how data is displayed*

Creating custom field widget

Folder Structure

Make sure your module has this structure:

```
realname_field/
├── realname_field.info.yml
├── src/
│   └── Plugin/
│       └── Field/
│           ├── FieldType/
│           │   └── RealName.php
│           └── FieldWidget/
│               └── RealNameDefaultWidget.php
```

Creating custom field widget

Define the Custom Field Widget Plugin
Create the file:
src/Plugin/Field/FieldWidget/RealNameDefaultWidget.php

Example Code

```
<?php

namespace Drupal\realname_field\Plugin\Field\FieldWidget;

use Drupal\Core\Field\WidgetBase;
use Drupal\Core\Field\FieldItemListInterface;
use Drupal\Core\Form\FormStateInterface;
```

Creating custom field widget


```
/**
 * Plugin implementation of the 'realname_default' widget.
 *
 * @FieldWidget(
 *   id = "realname_default",
 *   label = @Translation("Real Name (First + Last)"),
 *   field_types = {
 *     "realname"
 *   }
 * )
 */
class RealNameDefaultWidget extends WidgetBase {
```

Creating custom field widget


```
/**
  * {@inheritdoc}
  */

  public function formElement(FieldItemListInterface $items, $delta, array $element, array
&$form, FormStateInterface $form_state) {

    // Load existing values if available
    $first_name = isset($items[$delta]->first_name) ? $items[$delta]->first_name : '';
    $last_name = isset($items[$delta]->last_name) ? $items[$delta]->last_name : '';
```

Creating custom field widget


```
// Return a pair of text fields for first and last name
$element['first_name'] = [
  '#type' => 'textfield',
  '#title' => $this->t('First name'),
  '#default_value' => $first_name,
  '#size' => 20,
  '#maxlength' => 255,
  '#required' => TRUE,
];
```

Creating custom field widget


```
  $element['last_name'] = [
      '#type' => 'textfield',
      '#title' => $this->t('Last name'),
      '#default_value' => $last_name,
      '#size' => 20,
      '#maxlength' => 255,
      '#required' => TRUE,
    ];

    return $element;
  }

}
```

Creating custom field widget

Clear Cache
Rebuild Drupal's plugin cache so it recognizes the new widget:
drush cr or Configuration→performance→clear all caches

Use the Widget

Go to **Structure** → **Content types** → **Article** → **Manage fields**

Add or edit your **Real Name** field
Under **Widget**, you'll now see your custom widget:
"Real Name (First + Last)"

Creating custom field widget

Select it and save.

Now, when you edit a content item, the form will show **two text boxes** — one for **First name** and one for **Last name**.

THANK YOU

Department of Computer Applications